


# VALUE ADDED COURSE ON AI/ ML - TASK 5

## NAME: D SHARMILA

## USN: 24BBCAG009

## COURSE: BCA (GENERAL)

## SEMESTER: 5$^{TH}$ / YEAR: III

## SUBJECT: ARTIFICIAL INTELLIGENCE

# EXPERIMENT 5: Movie / E-Commerce Recommendation System using Content-Based Filtering

## AIM:

To develop a content-based movie recommendation system using TF-IDF vectorization and cosine similarity to recommend similar movies.

## SYSTEM REQUIREMENTS:

### Hardware Requirements

- **Processor:** Intel Core i3 or equivalent (minimum)
- **RAM:** 4 GB or higher
- **Internet Connection:** Required (for loading raw dataset directly in Google Colab)

### Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux
- **Platform:** Google Colab / Jupyter Notebook
- **Environment:** Python 3.x
- **Required Libraries:** pandas, numpy, scikit-learn

## PROCEDURE:

1. **Dataset Acquisition:** Load the TMDB movie dataset from the given URL using the Pandas library.

2. **Feature Extraction:** Extract important movie details such as title, genres, and overview from the dataset.

3. **Data Cleaning:** Handle missing overview values and extract genre names from the JSON-formatted data.

4. **Text Profile Construction:** Combine the movie overview and genre information into a single text profile called tags.

5. **TF-IDF Vectorization:** Convert the text profiles into numerical vectors using TfidfVectorizer, removing common English stop words.

6. **Cosine Similarity Calculation**: Calculate the similarity scores between movies using the cosine_similarity() function.

7. **Recommendation Generation:** Create a recommendation function that accepts a movie title, ranks similar movies, and displays the top 5 recommendations with their similarity scores.

8. **Testing:** Test the recommendation system using The Dark Knight and Avatar to verify that it recommends movies with similar content.

## PROGRAM:

```
# ============================================================================
# Experiment 5: Movie / E-Commerce Recommendation System
# Dataset: TMDB 5000 Movies Benchmark Dataset
# ============================================================================

import pandas as pd

import numpy as np

import json

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity


# Step 1: Load Benchmark Dataset (TMDB 5000 Movies CSV)

DATA_URL = "https://raw.githubusercontent.com/datasets/tmdb-movie-
metadata/master/tmdb_5000_movies.csv"

df = pd.read_csv(DATA_URL)


print("--- Step 1: Dataset Overview ---")

print(f"Total Movies: {df.shape[0]}\n")

print(df[['id', 'title', 'genres', 'overview']].head(3))

print("\n" + "="*60 + "\n")


# Step 2: Helper Function to Extract Genres from JSON Strings

def parse_genres(genres_str):

    try:

        genres_list = json.loads(genres_str)

        return ' '.join([g['name'] for g in genres_list])

    except:

        return ''


df['genre_tags'] = df['genres'].apply(parse_genres)
```

```
df['overview_clean'] = df['overview'].fillna('')

# Combine Metadata into a single text profile
df['tags'] = df['overview_clean'] + " " + df['genre_tags']

# Retain essential columns
df_movies = df[['title', 'tags']].copy()

# Step 3: TF-IDF Vectorization of Content Tags
tfidf = TfidfVectorizer(stop_words='english', max_features=5000)
tfidf_matrix = tfidf.fit_transform(df_movies['tags'])

print(f"TF-IDF Vector Matrix Shape: {tfidf_matrix.shape[0]} items x {tfidf_matrix.shape[1]} features")
print("="*60 + "\n")

# Step 4: Compute Pairwise Cosine Similarity Matrix
similarity_matrix = cosine_similarity(tfidf_matrix, tfidf_matrix)

# Index Mapping for Fast Title Lookup
indices = pd.Series(df_movies.index, index=df_movies['title'].str.lower()).drop_duplicates()

# Step 5: Top-N Recommendation Engine Function
def recommend_movies(title, top_n=5):
    query_title = title.lower()
    if query_title not in indices:
        print(f"Error: '{title}' not found in dataset index.")
        return

    idx = indices[query_title]
    if isinstance(idx, pd.Series):
        idx = idx.iloc[0]
```

```
    # Calculate similarity scores for all items
    sim_scores = list(enumerate(similarity_matrix[idx]))

    # Sort items based on highest similarity scores (excluding the query item itself)
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)[1:top_n+1]

    movie_indices = [i[0] for i in sim_scores]
    scores = [round(i[1], 4) for i in sim_scores]

    results = pd.DataFrame({
        'Recommended Title': df_movies['title'].iloc[movie_indices].values,
        'Similarity Score': scores
    })

    print(f"--- Top {top_n} Recommendations for: '{df_movies['title'].iloc[idx]}' ---")
    print(results.to_string(index=False))
    print("="*60 + "\n")

# Step 6: Test Recommendation System with Sample Queries
print("--- Step 2: Testing Recommendation System Output ---")
recommend_movies("The Dark Knight", top_n=5)
recommend_movies("Avatar", top_n=5)
```

## OUTPUT:

```
Dataset loaded successfully!
Total Movies: 4803
       id                                     title  \
0   19995                                    Avatar
1     285  Pirates of the Caribbean: At World's End
2  206647                                   Spectre

                                              genres  \
0  [{"id": 28, "name": "Action"}, {"id": 12, "nam...
1  [{"id": 12, "name": "Adventure"}, {"id": 14, "...
2  [{"id": 28, "name": "Action"}, {"id": 12, "nam...

                                            overview
0  In the 22nd century, a paraplegic Marine is di...
1  Captain Barbossa, long believed to be dead, ha...
2  A cryptic message from Bond’s past sends him o...

TF-IDF Matrix Shape: (4803, 5000)

Top 5 Recommendations for: The Dark Knight
The Dark Knight Rises | Similarity Score: 0.381
Batman Forever | Similarity Score: 0.2875
Batman Returns | Similarity Score: 0.2768
Batman: The Dark Knight Returns, Part 2 | Similarity Score: 0.249
Batman Begins | Similarity Score: 0.2191

Top 5 Recommendations for: Avatar
Apollo 18 | Similarity Score: 0.2455
Beowulf | Similarity Score: 0.2078
The Helix... Loaded | Similarity Score: 0.1981
The American | Similarity Score: 0.1954
The Adventures of Pluto Nash | Similarity Score: 0.1688
```

## RESULT:

Thus, the content-based movie recommendation system was successfully developed using Python, TF-IDF vectorization, and cosine similarity. The system processed movie genres and overviews, calculated similarity scores between movies, and generated the top 5 movie recommendations for the selected titles, The Dark Knight and Avatar.